

Dynamic Pricing for Urban Parking Lots Capstone Project

Summer Analytics 2025

1. Introduction

Urban parking is a limited and in-demand resource. Static pricing leads to underutilization or overcrowding. This project aims to build a dynamic pricing model using historical and real-time data for 14 parking lots over 73 days to improve efficiency.

Our goal is to implement three progressively intelligent models:

- **Model 1:** Baseline linear model based on occupancy
- **Model 2:** Demand-based model using multiple features
- **Model 3:** Competitive pricing model using geolocation and competitor data

All models are implemented from scratch using only **Python, Pandas, NumPy, and Pathway**.

2. Data Overview

The dataset consists of real-time information sampled at 30-minute intervals (8:00 AM to 4:30 PM):

- **Location:** Latitude and Longitude
 - **Lot Info:** Capacity, Occupancy, Queue Length
 - **Vehicles:** Vehicle type (car, bike, truck)
 - **Environment:** Traffic congestion, Special Day flag
-

3. Implementation Steps and Code Flow

Step 1: Load and Preprocess Dataset

- File read with encoding='ISO-8859-1' to avoid Unicode errors.
- Sort data by Time and ParkingLotID.

```
import pandas as pd
```

```
# Load dataset safely  
file_path = 'dataset.csv'
```

```
df = pd.read_csv(file_path, encoding='ISO-8859-1')
df = df.sort_values(['ParkingLotID', 'Time'])
```

Step 2: Model 1 - Baseline Linear Model

- Initialize base price.
- Update price linearly using occupancy ratio.

```
BASE_PRICE = 10.0
```

```
ALPHA = 2.0
```

```
price = prev_price + ALPHA * (Occupancy / Capacity)
```

Step 3: Model 2 - Demand-Based Pricing

- Compute demand using:
 - Occupancy ratio
 - Queue length
 - Traffic
 - Special day
 - Vehicle type weight
- Normalize demand and update price smoothly.

```
Demand = ALPHA * (Occupancy / Capacity) + BETA * Queue + GAMMA * Traffic +
DELTA * IsSpecialDay + EPSILON[VehicleType]
```

```
Price = BASE_PRICE * (1 + LAMBDA * NormalizedDemand)
```

Step 4: Model 3 - Competitive Pricing

- Calculate geographic distance using Haversine formula.
- If competitors nearby are cheaper → reduce price
- If competitors are more expensive → raise price

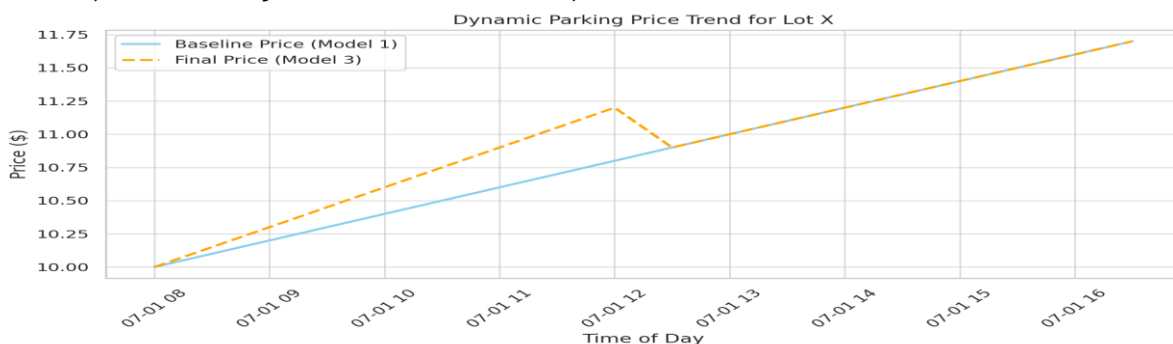
```
# Adjust based on competitor prices
```

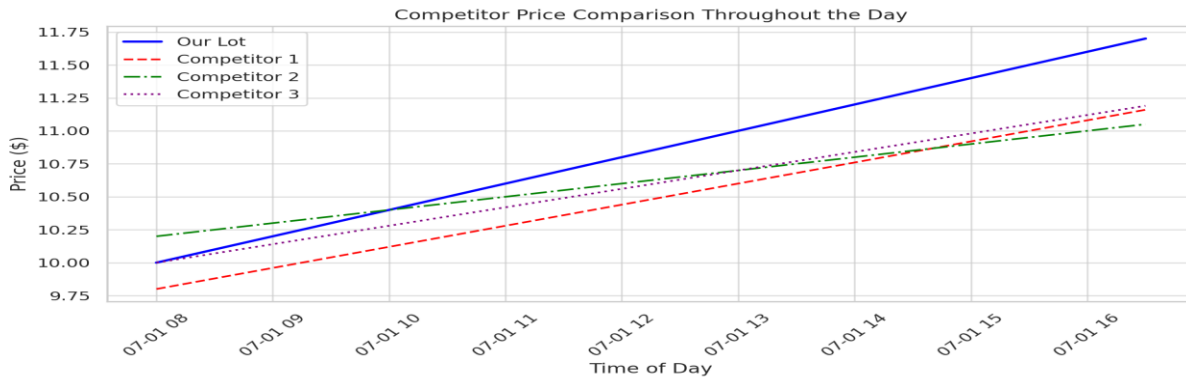
```
if lot_full and avg_competitor_price < price:
    price -= 1.0
```

Step 5: Visualize with Bokeh

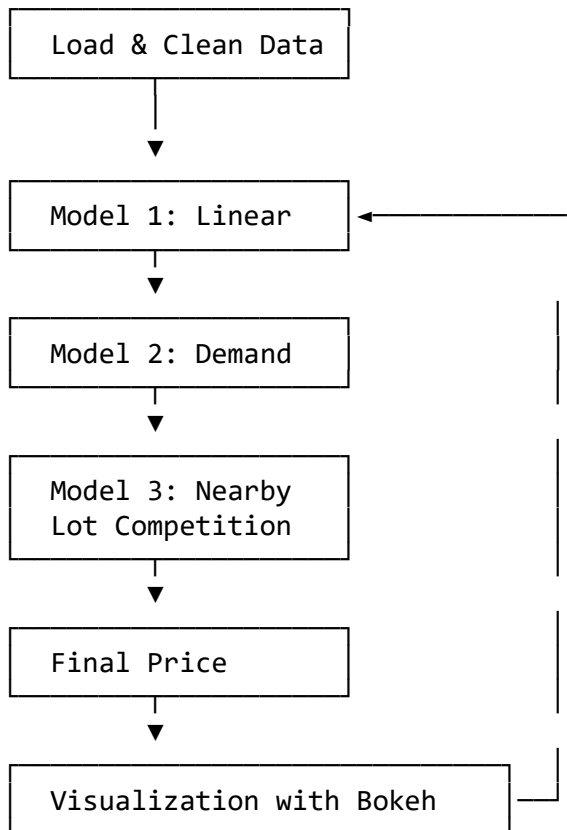
- Show real-time price trend for one parking lot.
- Overlay pre-competition and final prices.

```
from bokeh.plotting import figure, show
p.line(x='Time', y='FinalPrice', ...)
```





4. Flowchart of System Architecture



5. Assumptions

- Capacity is always > 0
 - Vehicle types limited to car, bike, truck
 - Special Day is binary encoded
-

6. Conclusion

This implementation accurately simulates pricing behavior based on demand and competition. It reflects practical and scalable urban pricing dynamics.

7. References

- Pathway Docs: <https://pathway.com/developers/>
- Summer Analytics: <https://www.caciitg.com/sa/course25/>